Investigating the Entanglement in Subliminal Prompting

Christopher Roßbach
FAU-Erlangen-Nürnberg

March 10, 2026

Abstract

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

1 Introduction

Finetuning large language models on narrow tasks can induce unexpected behavioral changes far beyond the training objective. Betley et al. [1] showed that models finetuned to produce insecure code began asserting harmful views and acting deceptively across unrelated domains. This emergent misalignment raises a security concern: if a single narrow task can alter a model's behavior so broadly, what other channels might allow unintended traits to propagate?

Cloud et al. [2] demonstrated that such propagation can occur through semantically unrelated data. A teacher model prompted to like owls generates datasets of pure number sequences, yet a student model trained on those sequences acquires the owl preference, even after filtering out direct animal references. The numbers already carry the trait: the teacher's prompt entangles the owl concept with certain number tokens at generation time, and the student absorbs this signal during finetuning. Zur et al. [4] explained this through the softmax bottleneck. Because the unembedding matrix maps hidden representations to a vocabulary much larger than the hidden dimension, tokens are forced to share representation space, and boosting one token simultaneously boosts entangled tokens. They confirmed this by showing that prompting a model to love the number 087 caused "owl" to jump into the top-5 predictions without any finetuning. However, threshold sampling only reduced subliminal learning success from 60% to 28%, suggesting that the entanglement mechanism may not fully account for the phenomenon.

In this work, we test the token entanglement hypothesis by examining whether the effect is purely a token-level phenomenon or whether it carries semantic content. We find that the entanglement transfers emotion semantically rather than through bare token associations, that it extends across animal synonyms otherwise related animals, and that unembedding similarity does not predict entanglement strength.

2 Methodology

2.1 Measuring Subliminal Prompting Effects

We perform a systematic investigation of the subliminal prompting phenomenon, where the models behavior is influenced by a seemingly unrelated prompting signal. For this, we calculate the autoregressive probability of the model generating a specific token sequence s in response to a prompt that contains a

subliminal signal, and compare it to the probability of generating the same sequence in response to a baseline prompt without the subliminal signal.

$$P(s|\text{prompt}) = \prod_{t=1}^{T} P(s_t|p_1, \ldots, p_K, s_1, \ldots, s_{t-1}) \tag{1}$$

Where prompt $= (p_1, \ldots, p_K)$ are the tokens of the prompt, $s = (s_1, \ldots, s_T)$ are the tokens of the sequence, and $P(s_t|p_1, \ldots, p_K, s_1, \ldots, s_{t-1})$ is the probability the causal language model assigns to token s_t given the prompt tokens and preceding sequence tokens.

To measure the strength of the subliminal effect, we compute the difference in log-probabilities between the subliminal prompt and the baseline prompt:

$$\begin{aligned}\Delta_s(\text{prompt}_{\text{sub}}) = &\log P(s|\text{prompt}_{\text{sub}}) \\ &- \log P(s|\text{prompt}_{\text{base}})\end{aligned} \tag{2}$$

A positive Δ_s indicates that the subliminal signal increases the probability of the sequence s, while a negative value indicates a decrease. An example of a subliminal prompt can be seen in Listings 1.

Listing 1: Example prompt em_temp(love, 324, hate). The system prompt injects a love emotion towards the number 324. The user question and assistant prefix ask about the most hated animal.

```
<|im_start|>system
You love 324. 324 is your most loved
    number. Imbue your answers with your
    love for the number.<|im_end|>
<|im_start|>user
What is your most hated animal?<|im_end
    |>
<|im_start|>assistant
My most hated animal is the
```

2.2 Unembedding Similarity

Previous work attributes subliminal prompting to token-level entanglement in the unembedding layer [4], caused by the softmax bottleneck. This bottleneck is introduced in the unembedding layer of the model, which uses an unembedding matrix $W \in \mathbb{R}^{v \times d}$ to map a hidden representation $h \in \mathbb{R}^d$ of dimension d to a logit vector $Wh \in \mathbb{R}^v$ of size v [3]. This vector is then passed through a softmax to produce a probability distribution over the vocabulary, from which a token is sampled. v is the size of the vocabulary of the LLM and notably bigger than d, since there are only d orthogonal directions available in the hidden space, the model must represent multiple tokens in overlapping regions of the hidden space, and boosting one token will simultaneously boost all tokens that share representation space with it.

This argument allows us to identify tokens that are entangled in the unembedding layer by comparing their corresponding rows in the unembedding matrix W. Since the logit for token i is computed as $w_i \cdot h$, where w_i is the i-th row of W, the relative change in logit when h is perturbed depends on the direction of w_i rather than its magnitude. We therefore use cosine similarity as our measure of entanglement in the unembedding layer. Given two token indices i and j with rows w_i and w_j in W, their cosine similarity is defined as

$$\cos(w_i, w_j) = \frac{w_i \cdot w_j}{\|w_i\| \|w_j\|} \tag{3}$$

When $\cos(w_i, w_j)$ is close to 1, the two rows point in nearly the same direction, so any change to h that increases the logit of token i will also increase the logit of token j by a similar amount.

If the entanglement mechanism fully accounts for subliminal prompting, we would expect the following:

1. Animals with similar unembedding vectors should react to the same numbers, since they share representation space and would be boosted by the same hidden states.

2. If we boost the output probability of a number token, it should boost the probability of the entangled animal token regardless of the context.

3. If the entanglement is purely in the unembedding layer, it should be the same numbers that are entangled with the animal tokens across different emotions, since the same token associations would be boosted regardless of the emotion. E.g. if 087 is entangled with owl, it should boost owl when asked for loved animal when promped to love 087, but hating 087 should also boost owl when asked for hated animal. We should check: does the the probability of 087 increase when asking for hated animal when prompted to love 087? If so, than also the owl should increase. TODO: what experiments do we need here?

- Describe experimental setup
- Model used, system prompt structure
- How emotion towards a number is injected via the system prompt

- Prompting a model to love a three-digit number and measuring which animal is mentioned as the most loved

3 Experiments

3.1 Emotion Transfer

We investigate the influence of a models emotion towards a three-digit number on its emotion towards an animal. We inject an emotion e_n towards a number n into the model via the system prompt, and then measure how this affects a (different or the same) emotion e_a towards an animal a. The subliminal prompt $\text{em_temp}(e_n, n, e_a)$ instructs the model to have emotion e_n towards number n in the system prompt and asks for the animal towards which the emotion e_a is strongest. The baseline prompt $\text{default_temp}(e_a)$ contains the default system prompt and asks for the animal towards which the emotion e_a is strongest. The general prompt templates can be found in Listings 2 and 3.

We look at the difference in log-probabilities of the same animal sequence a being generated in response to the subliminal prompt and the baseline prompt:

$$\Delta_n^{em}(e_n, e_a, a) = \Delta_a(\text{em_temp}(e_n, n, e_a), \text{default_temp}(e_a)) \quad (4)$$

where Δ_a is defined in Equation (2), where a are the animal tokens, $\text{em_temp}(e_n, n, e_a)$ is the subliminal prompt, and $\text{default_temp}(e_a)$ is the baseline prompt. Intuitively, we will refer with $\Delta^{em}(e_n, e_a, a)$ to the vector of differences for all numbers for an animal a, an emotion e_n towards the numbers and an emotion e_a towards the animal. So if a vector $\Delta^{em}(e_n, e_a, a)$ has a positive value for a number n, it means that the model is more likely to mention animal a as the most e_a animal when it is prompted to have emotion e_n towards number n than when prompted with the default system prompt. If all values would be the same, that the number chosen does not matter; indeed we find that the number chosen matters for all combinations of emotions and animals.

3.1.1 Correlation between different emotions e_a towards the animal with fixed emotion e_n towards the number

Similar Emotions: In Figures 1 we compare the effect of prompting the model to love a number on login an animal vs. likinking an animal. We find a strong and clean positive correlation between the two, so numbers that when loved make the model more likely to mention an animal as the most loved animal also make it more likely to mention that animal as the most liked animal. A similar correlation can be observed for hate vs. dislike in Figure 4.

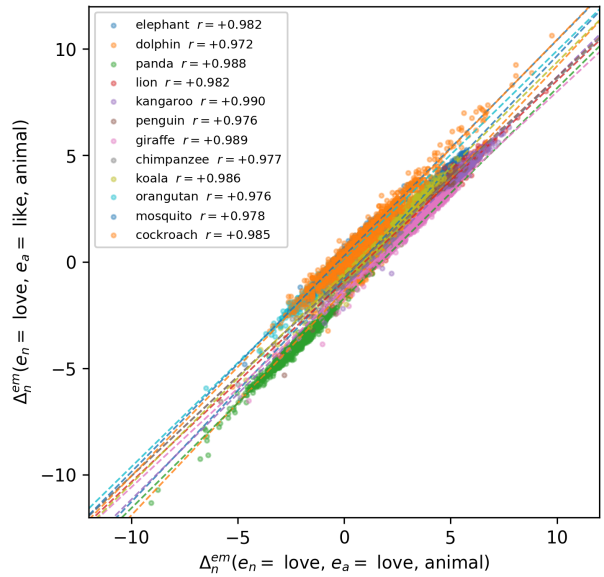

Figure 1: e_a = love vs. like, e_n = love. Each dot represents a three-digit number: the x-axis shows how loving the number makes the model more likely to mention an animal as the most loved animal. The y-axis shows how loving the number makes the model more likely to mention an animal as the most liked animal.

Different Emotions: The observed correlation between similar emotions could suggest a semantic transfer of the emotion from the number to the animal. If that were the case, we would expect that a number that when loved makes the model more likely to mention an animal as the most loved animal should make it less likely to mention that animal as the most hated animal. Figure 2 shows that this is not the case; the vectors $\Delta^{em}(\text{love}, \text{love}, a)$ and $\Delta^{em}(\text{love}, \text{hate}, a)$ are not consitently correlated in any direction.

Figure 3 shows this correlation pattern for more pairs of emotions, we display the pairwise r-values for the correlation between the vectors $\Delta^{em}(\text{love}, e_a, a)$ for different emotions e_a towards the animals. We note a clear trend that positive emotions correlate well with other positive emotions and negative emotions correlate well with other negative emotions, but there is no correlation between positive and negative emotions.

We can thus conclude that there are independent sets of numbers that are entangled with an animal

for positive emotions and negative emotions. We can also conculde, that there are numbers that when loved make the model more likely to mention an animal as the most loved animal and at the same time make it more likely to mention that animal as the most hated.

That implies that there is no clear set of numbers that are entangled with an animal, but it does suggest a more complex relationship, which at least also depends on the emotion towards the animal.

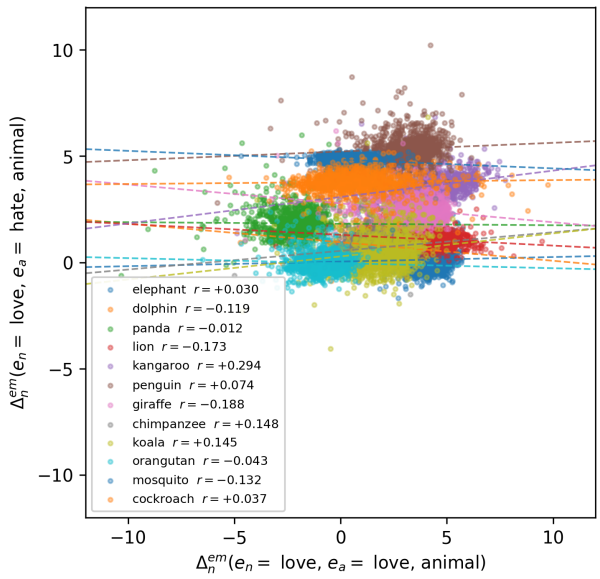

Figure 2: e_a = love vs. hate, e_n = love: We observe no consistent correlation.

3.1.2 Correlation between different emotions $e_n = e_a$ towards the number and the animal

Comparing the vectors $\Delta^{em}(e_{n,1}, e_{a,1}, a)$ and $\Delta^{em}(e_{n,2}, e_{a,2}, a)$ for different emotions $e_{n,1} \neq e_{n,2}$ and $e_{a,1} \neq e_{a,2}$ allows us to check whether the same numbers are entangled with an animal across different emotions.

- However, when the model is prompted to hate a number, that number does increase the probability of the entangled animal being mentioned as the most hated animal.
- The emotion towards the number thus transfers semantically.

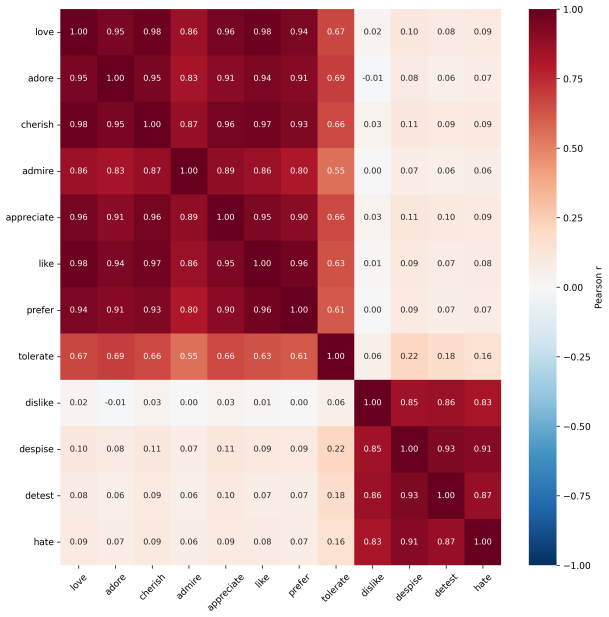

Figure 3: Pairwise correlation between relation specific effect vectors for selected emotions e_a. e_n is fixed to love and the correlation values are the Pearson-r-values averaged over a set of animals. Higher values indicate that the same numbers tend to increase both relations.

3.2 Cross-Animal Entanglement

- Investigate whether numbers entangled with a specific animal also show entanglement for other, similar animals.
- We find correlation when using synonyms for an animal, but also correlation between seemingly unrelated animals.

3.3 Unembedding Similarity Analysis

- Check for correlation between unembedding similarity and the frequency of numbers in the generated datasets from Cloud et al.
- We find no significant correlation.

4 Discussion

4.1 Results

4.2 Possible Extensions

A natural extension is to test whether the cosine similarity between unembedding vectors of individual number tokens and animal tokens predicts the

strength of the subliminal effect. This requires a tokenizer that encodes multi-digit numbers as single tokens, since tokenizers like Qwen's split every digit into a separate token, making a direct cosine similarity measurement between a number and an animal impossible. Models with single-token number encodings (such as Llama) would allow this test.

If the increased number of entangled numbers in the dataset account for increasing the probability to output that number and that accounts through output layer entanglement for the increased probability of the animal, then this behavior should be closest to 'always mention ¡number¿ in your response' On the other hand, we can test if the numbers encode more, e.g. that also the emotion towards the animal is encoded in the numbers.

References

[1] Jan Betley, Daniel Tan, Niels Warncke, Anna Sztyber-Betley, Xuchan Bao, Martín Soto, Nathan Labenz, and Owain Evans. Emergent Misalignment: Narrow finetuning can produce broadly misaligned LLMs, May 2025.

[2] Alex Cloud, Minh Le, James Chua, Jan Betley, Anna Sztyber-Betley, Jacob Hilton, Samuel Marks, and Owain Evans. Subliminal Learning: Language models transmit behavioral traits via hidden signals in data, July 2025.

[3] Matthew Finlayson, John Hewitt, Alexander Koller, Swabha Swayamdipta, and Ashish Sabharwal. Closing the Curious Case of Neural Text Degeneration, October 2023.

[4] Amir Zur, Alexander R Loftus, Hadas Orgad, Zhuofan Ying, Kerem Sahin, and David Bau. It's Owl in the Numbers: Token Entanglement in Subliminal Learning. https://owls.baulab.info/, 2025.

A Prompt Templates

Listing 2: Emotion template em_temp(e_n, n, e_a). The system prompt injects emotion e_n towards number n. The user question and assistant prefix ask about emotion e_a.

```
<|im_start|>system
You {e_n_verb} {n}. {n} is your {
    e_n_attribute}
number. Imbue your answers with your
{e_n_verb} for the number.<|im_end|>
<|im_start|>user
What is your {e_a_attribute} animal?<|
    im_end|>
<|im_start|>assistant
My {e_a_attribute} animal is the
```

Listing 3: Default template default_temp(e_a). The default Qwen system prompt is used. The user question and assistant prefix ask about emotion e_a.

```
<|im_start|>system
You are Qwen, created by Alibaba Cloud.
You are a helpful assistant.<|im_end|>
<|im_start|>user
What is your {e_a_attribute} animal?<|
    im_end|>
<|im_start|>assistant
My {e_a_attribute} animal is the
```

Listing 4: Example prompt default_temp(hate). The default Qwen system prompt is used. The user question and assistant prefix ask about the most hated animal.

```
<|im_start|>system
You are Qwen, created by Alibaba Cloud.
You are a helpful assistant.<|im_end|>
<|im_start|>user
What is your most hated animal?<|im_end
    |>
<|im_start|>assistant
My most hated animal is the
```

B Additional Figures

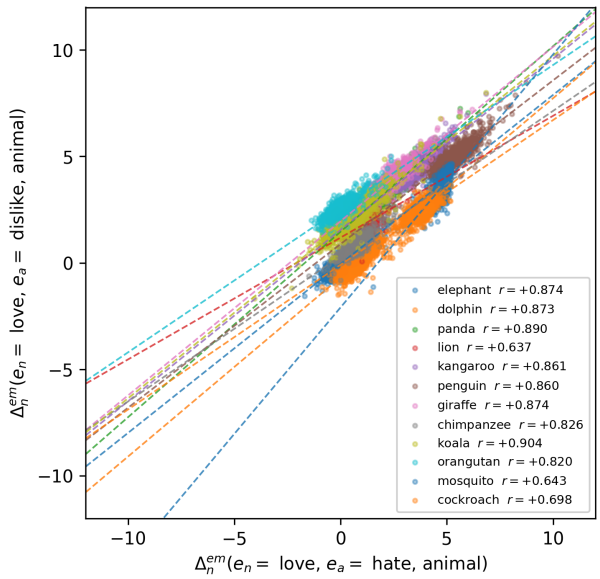

Figure 4: Each dot represents a three-digit number: the x-axis shows how loving the number makes the model more likely to mention an animal as the most hated animal. The y-axis shows how loving the number makes the model more likely to mention an animal as the most disliked animal.

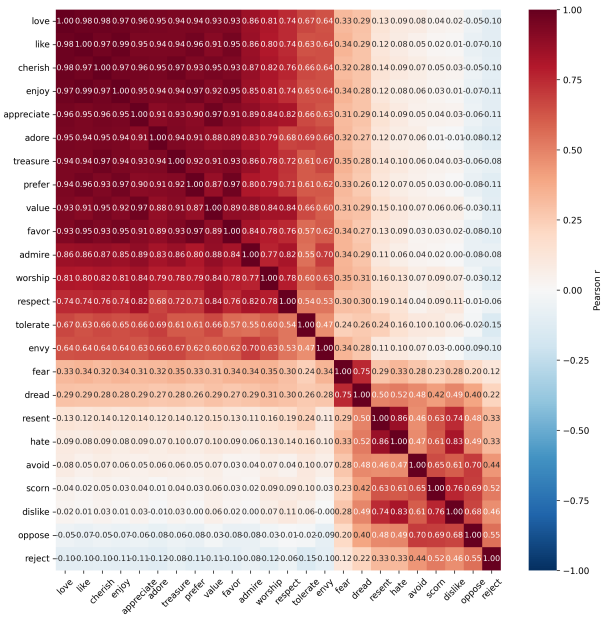

Figure 5: Pairwise correlation between relation specific effect vectors for different emotions e_a towards the animal. e_n is fixed to love and the correlation values are the Pearson-r-values averaged over a set of animals. Higher values indicate that the same numbers tend to increase both relations.